16. Reflection Overview (and When Not to Use It)

16. Reflection Overview (and When Not to Use It)

Phase: Phase 2 — Core language depth
Estimated time: 1 day
Slug: `16-reflection`

Learning objectives

- Explain reflect.Type and reflect.Value at a high level
- Recognize frameworks that use reflection (encoding/json)
- Prefer codegen or generics over heavy reflection
- Measure cost before using reflection in hot paths

Prerequisites

Complete prior numbered lessons. Type every example yourself.

What reflection does

The `reflect` package inspects types and values at runtime — powerful and easy to abuse.

```
func KindOf(v any) string {
    return reflect.TypeOf(v).Kind().String()
}
```

`encoding/json` uses reflection to map structs to JSON. That convenience costs CPU and clarity.

Prefer

- Generics for type-safe containers
- Interfaces for behavior
- `go generate` / codegen for repetitive glue

Use reflection for marshaling, debugging tools, and DI frameworks — not business logic.

Common mistakes (especially from Python instincts)

- Building your whole domain model on map[string]any + reflect
- Ignoring panic risks from invalid reflect usage
- Premature “flexible” APIs that erase types

Exercises

Work in `exercises/16-reflection/`. Read that folder’s `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/16-reflection/` until you have an honest failing attempt.

Self-check

1. Name one stdlib package that uses reflection
2. Why are generics often better?
3. What is a hot-path risk of reflect?

Further reading

- https://go.dev/blog/laws-of-reflection
- https://pkg.go.dev/reflect